



Industrial Machine Learning on Hadoop and Spark

Maks Nakhodnov, Bremen 2025

Distributed computing. Algorithms

Plan:

How to process big data? MR Algorithms:

- Distinct values, Cross-Correlation
- Relational functions
- TF-IDF
- Graph Processing

MapReduce. Distinct Values

Count the number of unique values for each group.

Note: #values >> #groups

i.e., user-site interaction —>

values == users, groups == pages

Record 1: F=1, G={a, b}
Record 2: F=2, G={a, d, e}
Record 3: F=1, G={b}
Record 4: F=3, G={a, b}

Result:

a -> 3 // F=1, F=2, F=3
b -> 2 // F=1, F=3
d -> 1 // F=2
e -> 1 // F=2

MapReduce. Distinct Values I

The single-phase solution:

1. Use an associative array to store unique values
2. Need to use a single reducer or perform post-aggregation (feasible as the number of categories is small)
3. Requires storing and sorting all groups in memory — may be infeasible

```
def map(f, [g1, g2, ...]):  
    for g in [g1, g2, ...]:  
        Emit(f, g)  
  
H = defaultdict(int)  
  
def reduce(f, [g1, g2, ...]):  
    [gn1, gn2, ...] = ExcludeDuplicates([g1, g2, ...])  
    for g in [gn1, gn2, ...]:  
        H[g] += 1  
  
for g in H:  
    Emit(g, H[g])
```

Record 1: F=1, G={a, b}
Record 2: F=2, G={a, d, e}
Record 3: F=1, G={b}
Record 4: F=3, G={a, b}

Result:

a -> 3 // F=1, F=2, F=3
b -> 2 // F=1, F=3
d -> 1 // F=2
e -> 1 // F=2

MapReduce. Distinct Values II

The two-phase solution:

1. Get unique (G, F) pairs
2. Count the number of F for each G

More complex to implement but more effective

```
# Distinct Values II. Stage 1
def map(f, [g1, g2, ...]):
    for g in [g1, g2, ...]:
        Emit((g, f), 1)

def reduce((g, f), [n1, n2, ...]):
    Emit((g, f), None)

# Distinct Values II. Stage 2
def map((g, f), _):
    Emit(g, 1)

def reduce(g, [n1, n2, ...]):
    Emit(g, sum([n1, n2, ...]))
```

Record 1: F=1, G={a, b}
Record 2: F=2, G={a, d, e}
Record 3: F=1, G={b}
Record 4: F=3, G={a, b}

Result:

a -> 3 // F=1, F=2, F=3
b -> 2 // F=1, F=3
d -> 1 // F=2
e -> 1 // F=2

MapReduce. Cross-Correlation I

Count the cooccurrences for every pair in the set of tuples:

$$\{(c_1, c_2, \dots, c_{t_n})\} \Rightarrow \{\# \text{cooccurrences } (c_i, c_j) | (c_i, c_j)\}$$

Simple, but network-heavy solution ($O(n^2)$ pairs to sort):

```
def map(_, [i1, i2, ...]):  
    for i in [i1, i2, ...]:  
        for j in [i1, i2, ...]:  
            Emit((i, j), 1)  
  
def reduce((i, j), [n1, n2, ...]):  
    Emit((i, j), sum([n1, n2, ...]))
```


MapReduce. Cross-Correlation II

Count the cooccurrences for every pair in the set of tuples:

$$\{(c_1, c_2, \dots, c_{t_n})\} \Rightarrow \{\# \text{cooccurrences } (c_i, c_j) | (c_i, c_j)\}$$

Mapper: $\begin{array}{l} (a, b) \rightarrow 1 \\ (a, c) \rightarrow 2 \\ (a, d) \rightarrow 5 \Rightarrow a \rightarrow \{b: 1, c: 2, d: 5, e: 3, f: 2\} \\ (a, e) \rightarrow 3 \\ (a, f) \rightarrow 2 \end{array}$

Reducer: $\begin{array}{l} + \\ a \rightarrow \{b: 1, \quad d: 5, e: 3\} \\ a \rightarrow \{b: 1, c: 2, d: 2, \quad f: 2\} \\ \hline a \rightarrow \{b: 2, c: 2, d: 7, e: 3, f: 2\} \end{array}$

Advantages:

- Sort & shuffle performed more effectively
- Can be used with Combiner

Disadvantages:

- More complex implementation
- Need to manage Mapper/Reducer RAM
- More complex objects to be sent

```
def map(_, [i1, i2, ...]):  
    for i in [i1, i2, ...]:  
        H = defaultdict(int)  
        for j in [i1, i2, ...]:  
            H[j] += 1  
        Emit(i, H)  
  
def reduce(i, [H1, H2, ...]):  
    H = MergeSum([H1, H2, ...])  
    for j in H:  
        Emit((i, j), H[j])
```

MapReduce. Relational Functions. Sets

Selection

```
def map(key, value):  
    if predicate(value):  
        Emit(value, None)
```

Projection

```
def map(key, value):  
    Emit(project(value), None)
```

```
def reduce(value, [...]):  
    Emit(value, None)
```

Union

```
def map(key, value):  
    Emit(value, None)
```

```
def reduce(value, [...]):  
    Emit(value, None)
```

Intersection

```
def map(key, value):  
    Emit(value, None)
```

```
def reduce(value, [n1, ...]):  
    if len([n1, ...]) == 2:  
        Emit(value, None)
```

Difference

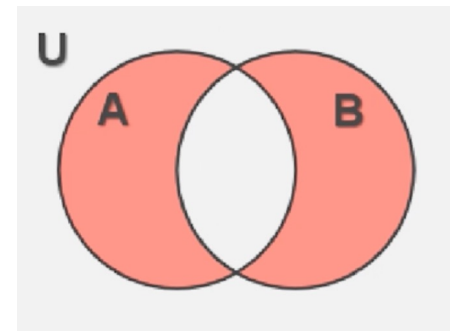
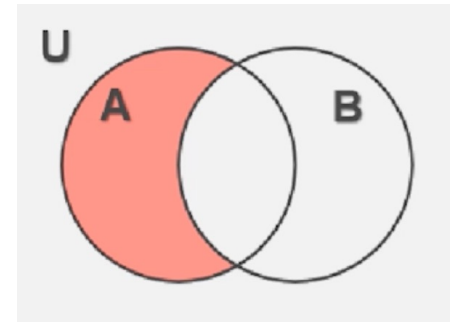
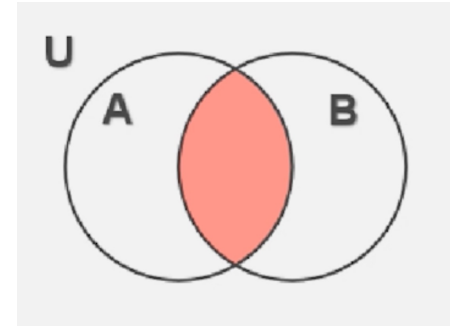
```
def map(key, value):  
    Emit(value, value.SetName)
```

```
# [n1, ...] can be ['A'], ['B'], ['A', 'B'], ['B', 'A']  
def reduce(value, [n1, ...]):  
    if len([n1, ...]) == 1 and n1 == 'A':  
        Emit(value, None)
```

Symmetric Difference

```
def map(key, value):  
    Emit(value, value.SetName)
```

```
# [n1, ...] can be ['A'], ['B'], ['A', 'B'], ['B', 'A']  
def reduce(value, [n1, ...]):  
    if len([n1, ...]) == 1 and n1 in ['A', 'B']:  
        Emit(value, None)
```



MapReduce. Repartition Join I

id ₁ : a		id ₁ : 5		id ₁ : (a, 5)
id ₂ : b	join	id ₂ : 7	=	id ₂ : (b, 7)
id ₂ : c		id ₂ : 4		id ₂ : (b, 4)
				id ₂ : (c, 7)
				id ₂ : (c, 4)

- Need to manage Reducer RAM
- For each key, we have to store all corresponding values
- Will fail if one of the sets is large

```
# Repartition Join
def map(_, (key, value)):
    # tag is either 'A' or 'B'
    Emit(key, (tag, value))

def reduce(key, [t1, ...]):
    H = defaultdict(list)
    for t in [t1, ...]:
        H[t.tag].append(t.value)
    for a in H['A']:
        for b in H['B']:
            Emit(None, [key, a, b])
```

MapReduce. Repartition Join II

id ₁ : a		id ₁ : 5		id ₁ : (a, 5)
id ₂ : b	join	id ₂ : 7	=	id ₂ : (b, 7)
id ₂ : c		id ₂ : 4		id ₂ : (b, 4)
				id ₂ : (c, 7)
				id ₂ : (c, 4)

- Load the small set in each mapper
- Use in-mapper combiner
- If one of the sets is small, we can use a **broadcast join**

```
# Replicated Join
H = dict()
A = load_small_set()
for key in A:
    H[key].append(A[key])

def map(_, (key, value)):
    for a in H[key]:
        Emit(None, (key, a, value))
```

MapReduce. TF-IDF

TF-IDF: Term Frequency — Inverse Document Frequency

- Still useful for Information Retrieval tasks
- Simplest text vectorization
- For each pair (word, document), give its importance
- IDF penalizes the frequent words
- TF encourages words that appear frequently in the document

$$\text{TF}(w, d) = \frac{|w \in d|}{|d|} \quad \text{IDF}(w, D) = \log \frac{|D|}{|w \in d_i|} \quad \text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w, D)$$

MapReduce. TF-IDF. Stage I. TF

Mapper:

- Input: (doc_name, doc_content)
- Output: ((word, doc_name), 1)

Reducer:

- Sums the values from the mapper
- Outputs: ((word, doc_name), tf)

Combiner: same as the Reducer

```
# TF
def map(doc_name, doc_content)
    for word in doc_content:
        Emit((word, doc_name), 1)

def reduce((word, doc_name), [v1, ...])
    Emit((word, doc_name), sum([v1, ...]))
```

MapReduce. TF-IDF. Stage II. $|w \in d_i|$

Mapper:

- Input: ((word, doc_name), tf)
- Output: (word, (doc_name, tf, 1))

Reducer:

- Sums ones from the mapper
- Output: ((word, doc_name), (tf, n))

```
# |w \in d_{i}|  
def map(word, doc_name, tf):  
    Emit(word, (doc_name, tf, 1))  
  
def reduce(word, [V1, ...])  
    n = sum([v for d, tf, v in V1, ...])  
    for (d, tf, _) in [V1, ...]:  
        Emit((word, doc_name), (tf, n))
```

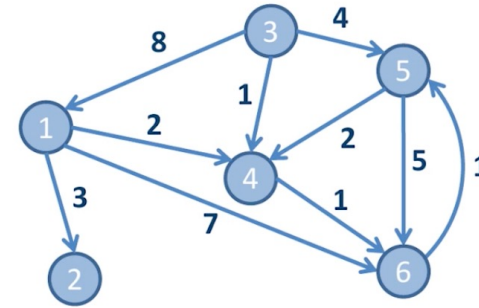
MapReduce. TF-IDF. Stage III

```
# TF * IDF  
def map((word, doc_name), (tf, n)):  
    Emit((word, doc_name), tf * log(N / n))
```


MapReduce. Graph processing

$G = (V, E)$

- V is a set of vertices (nodes)
- E is a set of edges (links)
- Each edge or vertex can contain additional information



$V = \{1, 2, 3, 4, 5, 6\}$

$E = \{[1,2], [1,4], [1,6], [3,1], \dots\}$

$W_{1,2} = 3, W_{1,4} = 2, W_{1,6} = 7, W_{3,1} = 8, \dots$

Examples of large graphs:

- Social graph
- Network graph
- Road graph
- ...

MapReduce. Graph processing

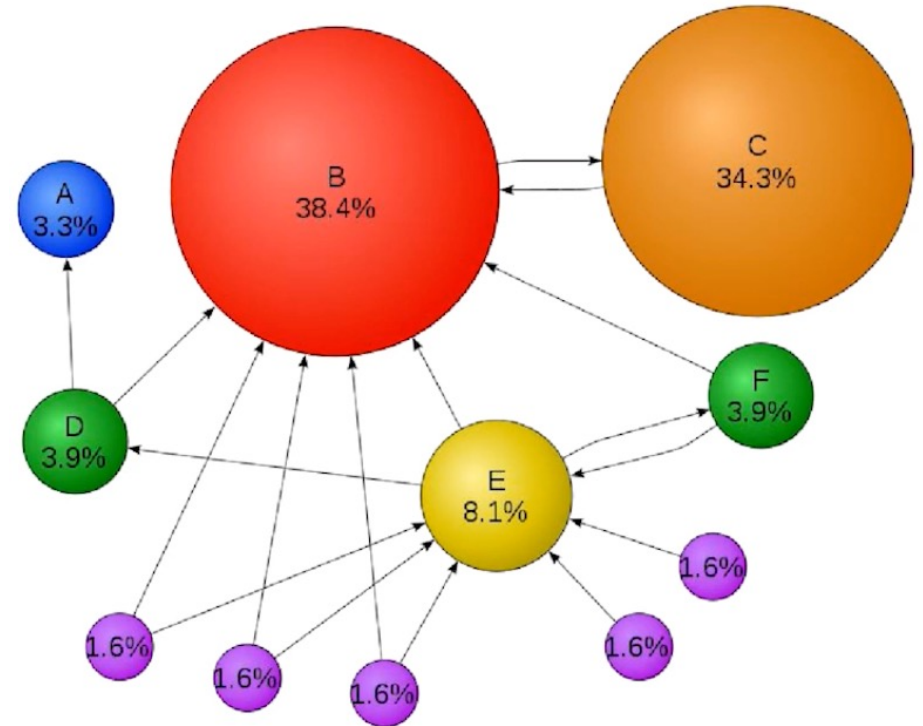
The common algorithms:

1. Graph traversal

- The shortest path (between two nodes or between all nodes)
- Minimum Spanning Tree
- Max Flow

2. Property evaluation

- Web pages ranking
 - PageRank
 - HITS



MapReduce. Graph processing

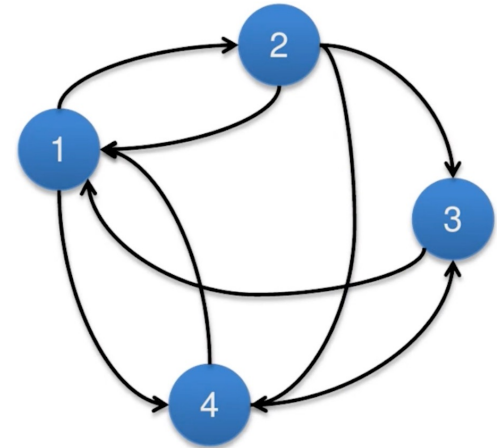
Key questions:

- How to store a graph in MapReduce?
 - Have to partition a graph somehow
- How to traverse a graph in MapReduce?
 - Have to process each chunk independently and then merge them back

MapReduce. Adjacency matrix

- + Easy to create and manage
- + Easy to get inbound and outbound edges for any vertex
- + Can be used for algebraic graph algorithms
- Requires $O(|V|^2)$ of storage
- Sparse representation

	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



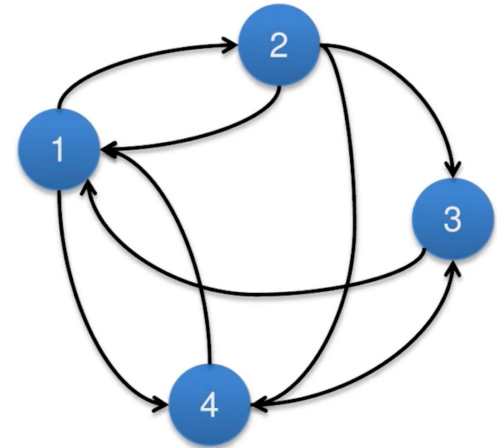
MapReduce. Adjacency list

- + An effective way to store a graph
- + Easy to get outbound edges for any vertex
- Hard to get inbound edges for any vertex

	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	1	0	0	0
4	1	0	1	0



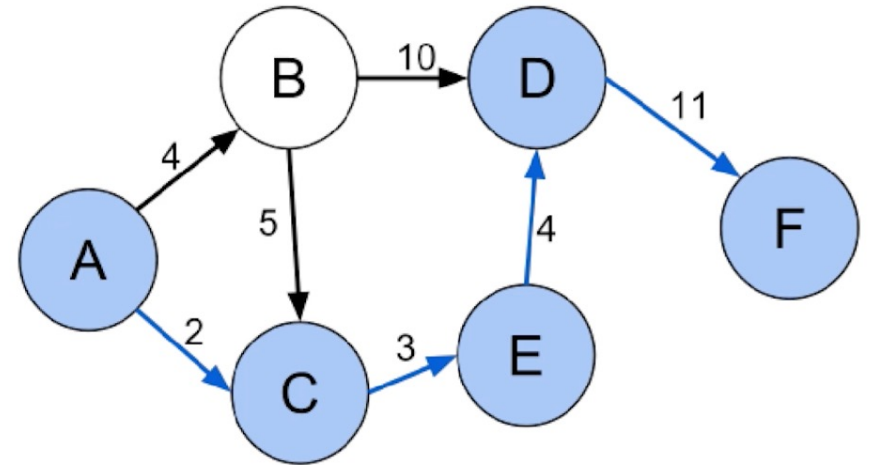
1: 2, 4
2: 1, 3, 4
3: 1
4: 1, 3



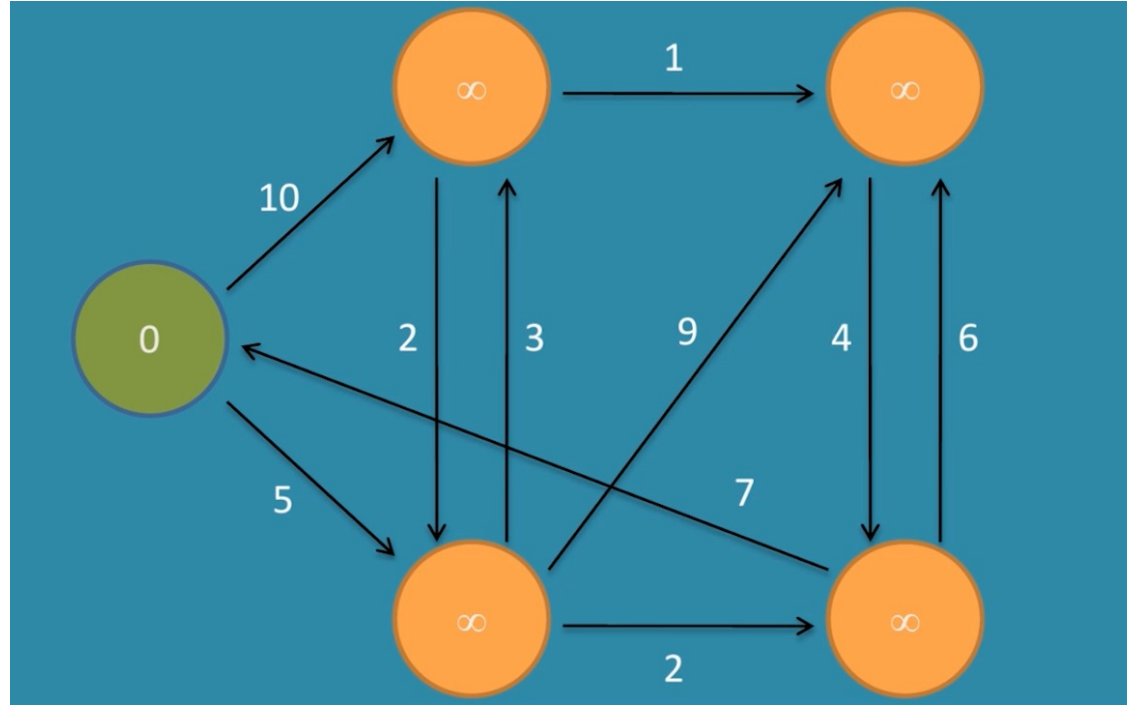
MapReduce. The shortest path

Find the shortest path between two nodes.

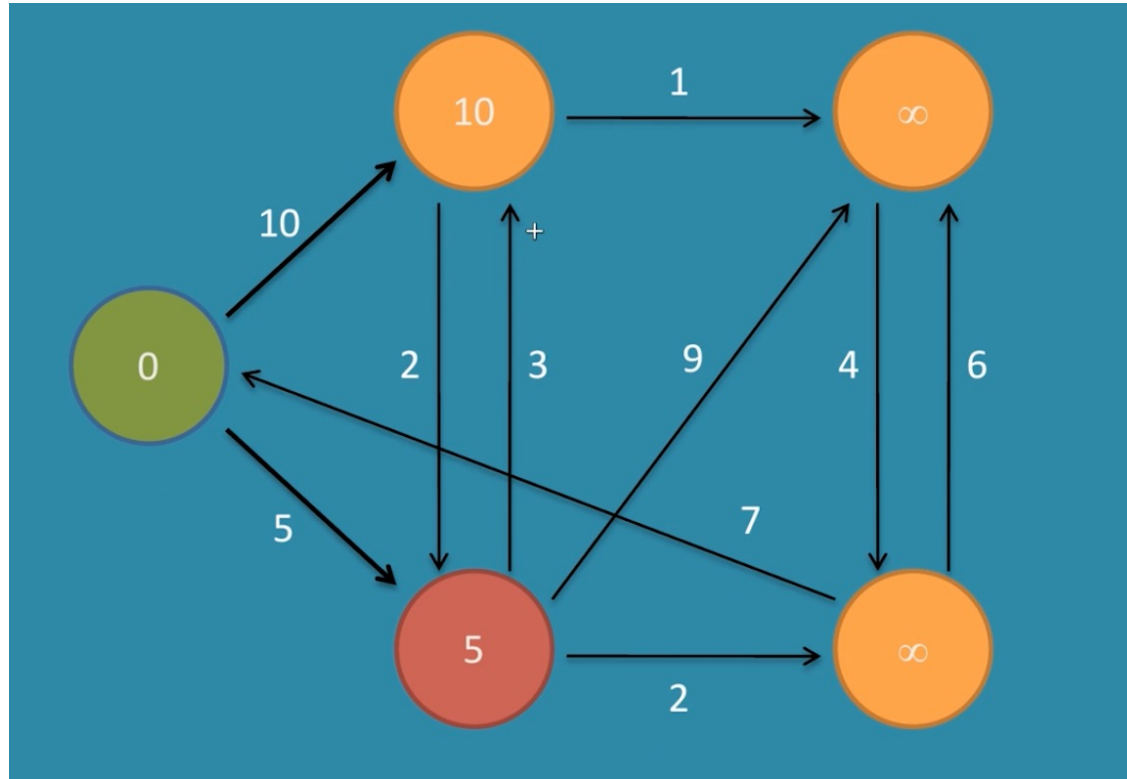
Assume all edges have non-negative cost.



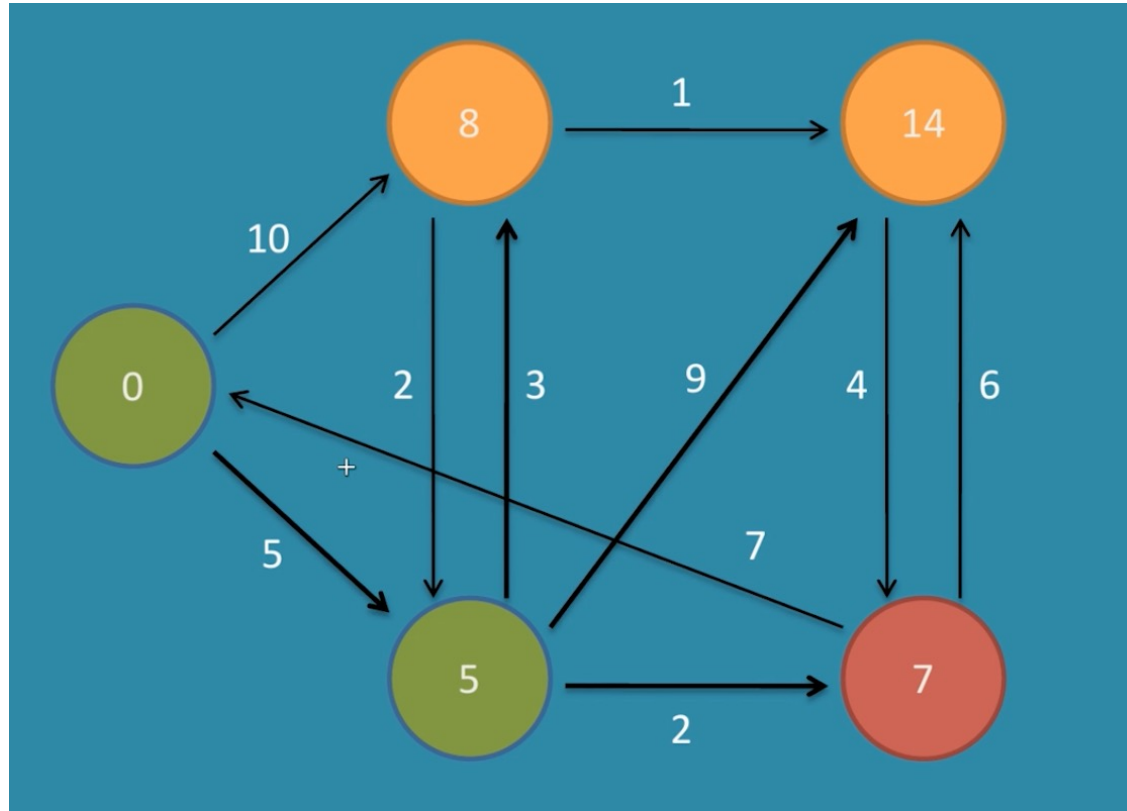
MapReduce. Dijkstra's algorithm. Example



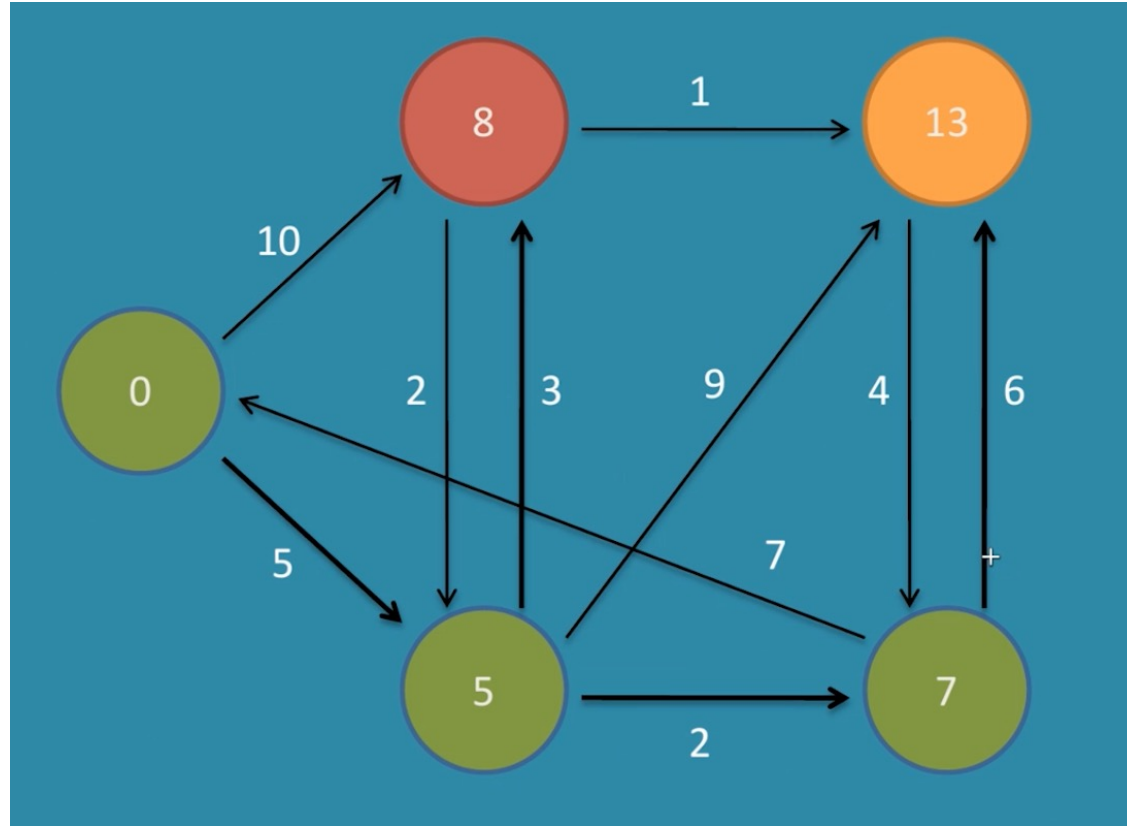
MapReduce. Dijkstra's algorithm. Example



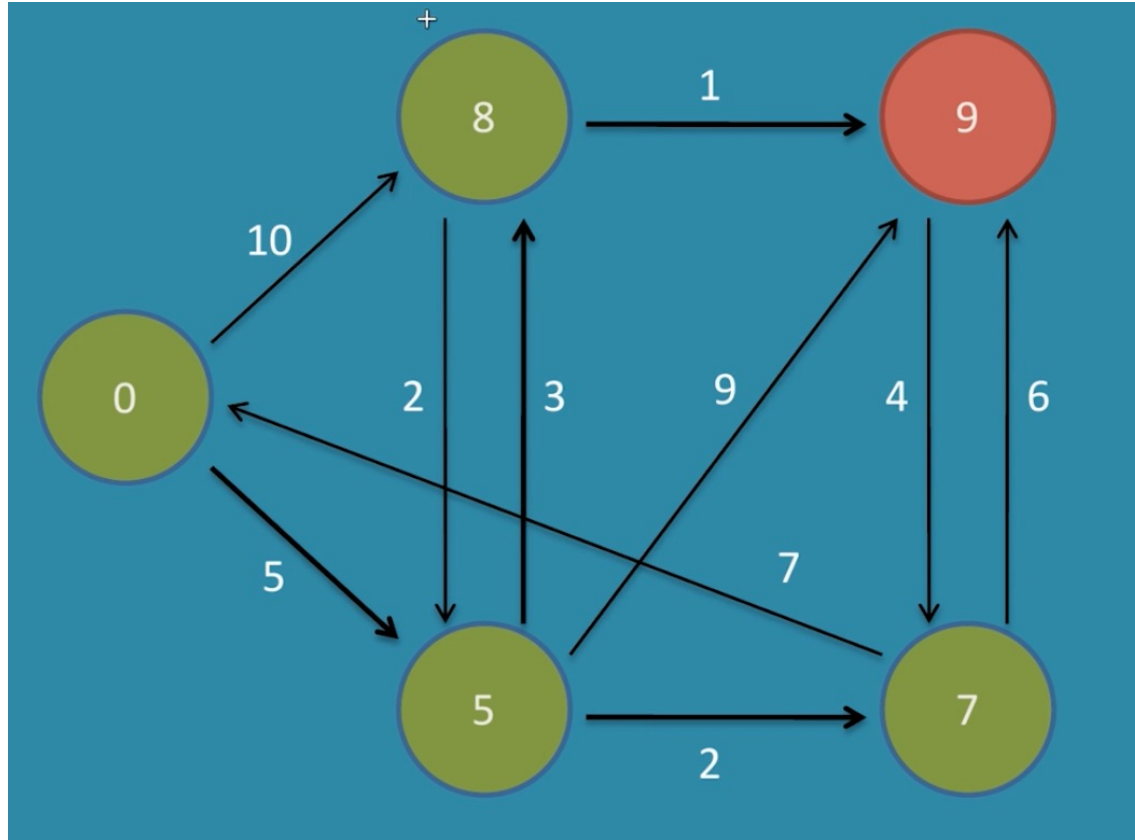
MapReduce. Dijkstra's algorithm. Example



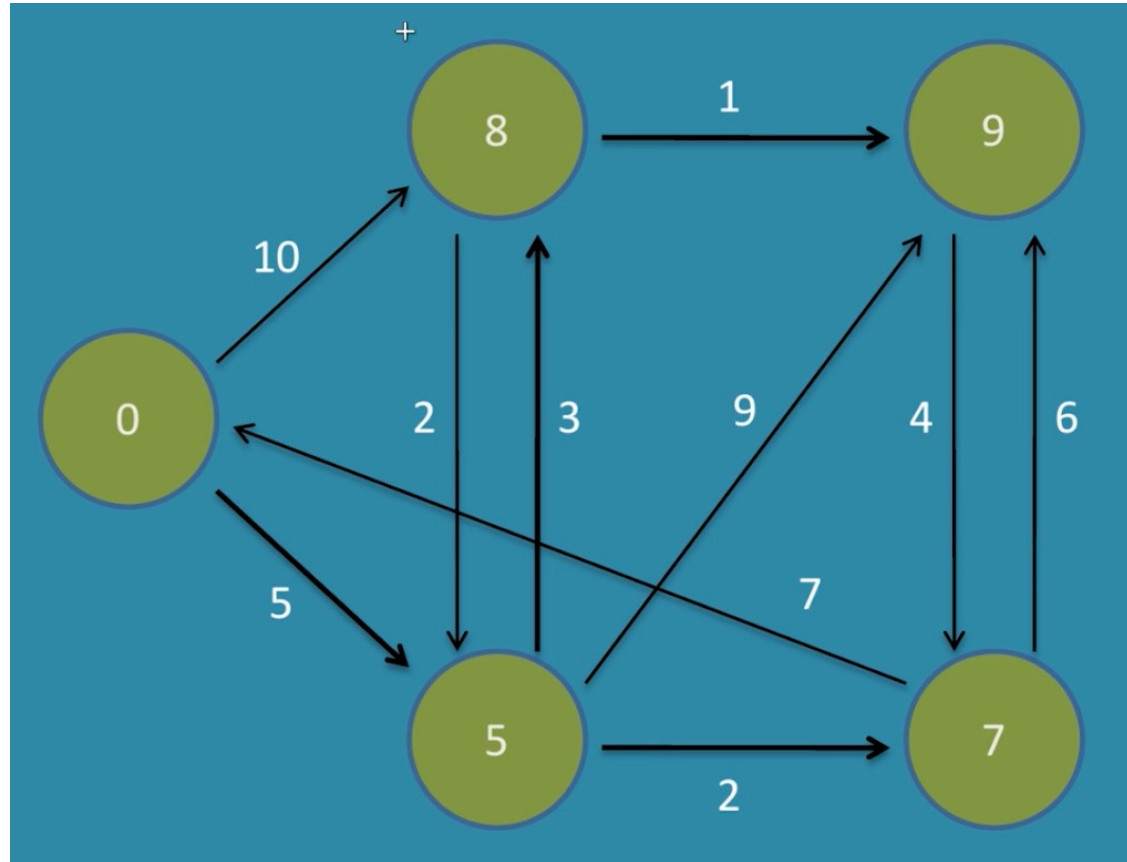
MapReduce. Dijkstra's algorithm. Example



MapReduce. Dijkstra's algorithm. Example



MapReduce. Dijkstra's algorithm. Example



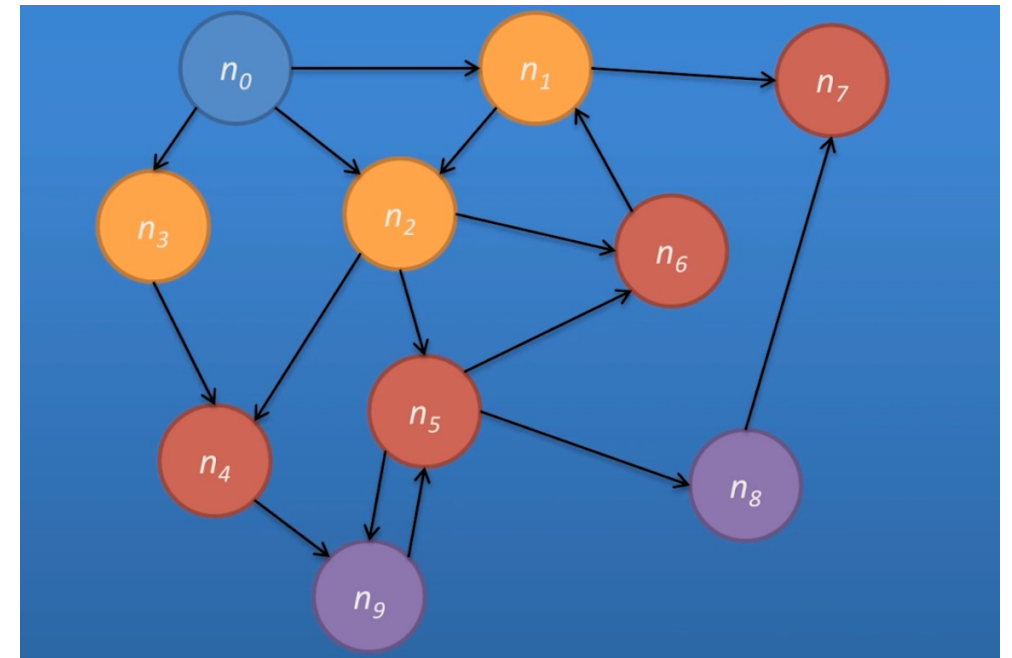
MapReduce. Dijkstra's algorithm

- Requires the global priority queue
- Hard to parallelize

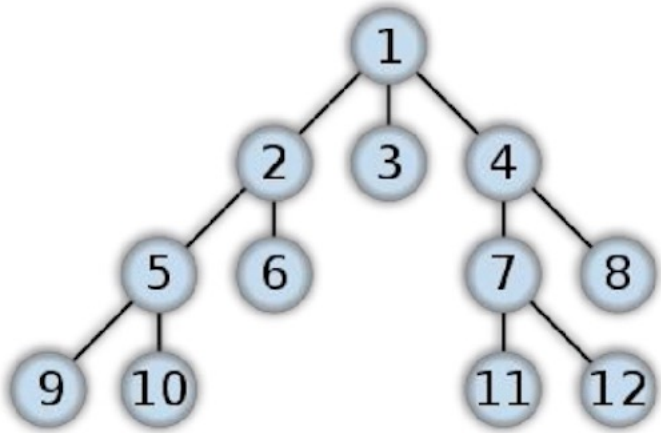
```
Dijkstra( $V$ ,  $s$ ,  $w$ )  
  for all vertex  $v \in V$  do  
     $d[v] \leftarrow \infty$   
   $d[s] \leftarrow 0$   
   $Q \leftarrow \{V\}$   
  while  $Q \neq \emptyset$  do  
     $u \leftarrow \text{ExtractMin}(Q)$   
    for all vertex  $v \in u.\text{AdjacencyList}$  do  
      if  $d[v] > d[u] + w(u, v)$  then  
         $d[v] \leftarrow d[u] + w(u, v)$ 
```

MapReduce. Distributed BFS

- Assume all weights are equal to one
- BFS will find the shortest path when we first hit the target node
- Each iteration of BFS propagates the frontier of nodes with known shortest paths



MapReduce. Distributed BFS



1 -> [0, {2, 3, 4}]

2 -> [∞ , {5, 6}]

3 -> [∞ , {}]

4 -> [∞ , {7, 8}]

5 -> [∞ , {9, 10}]

...

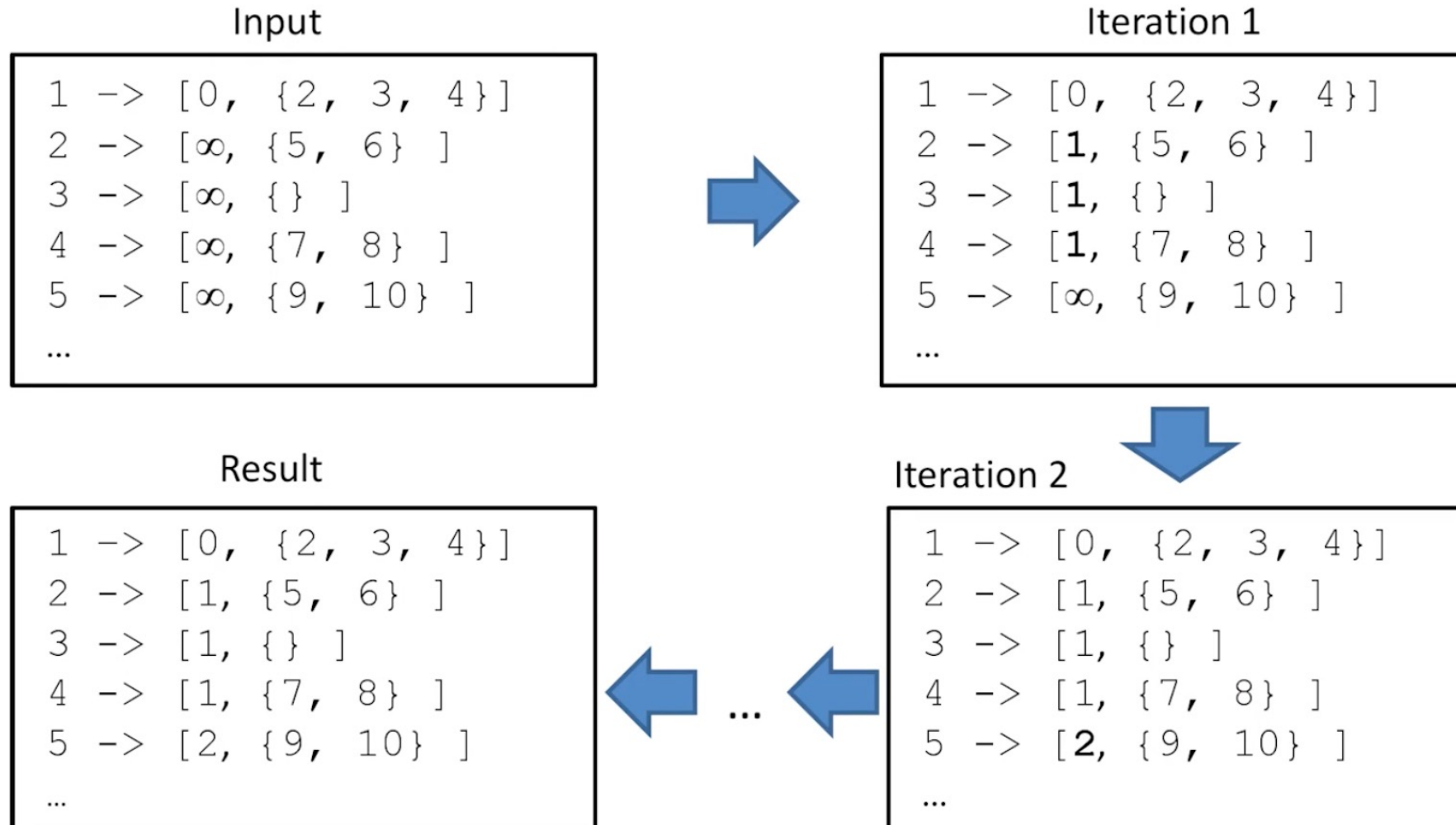
BFS

```
def map(node_id, (min_distance, adj_list)):
    # Preserve the structure (scheme) of the data
    Emit(node_id, adj_list)
    for node_jd in adj_list:
        Emit(node_jd, min_distance + 1)

def reduce(node_id, [D1, ...])
    min_distance, adj_list = -inf, None
    for d in [D1, ...]:
        if isinstance(d, list):
            adj_list = d
        elif d < min_distance:
            min_distance = d

    Emit(node_id, (min_distance, adj_list))
```


MapReduce. Distributed BFS



MapReduce. Distributed BFS

How to define a stopping criterion?

1. The first visit to the target node
2. The number of iterations == graph diameter

Can use:

- External driver to check if the criterion is met
 - Use Hadoop MapReduce Counters
-
- Less effective than Dijkstra's algorithm
 - Need to preserve the scheme of the graph
 - Useful for large graphs

MapReduce. Distributed BFS. Weighted Edges

Emit ($\text{min_distance} + \text{edge_distance}$)
instead of ($\text{min_distance} + 1$) in Mapper

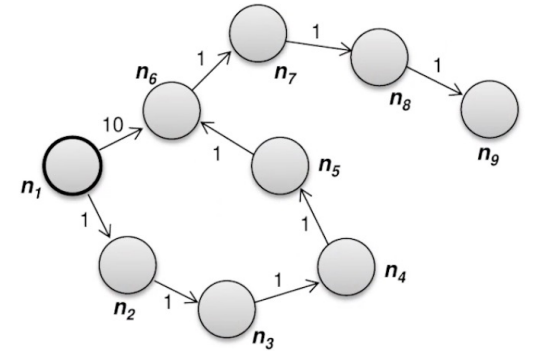
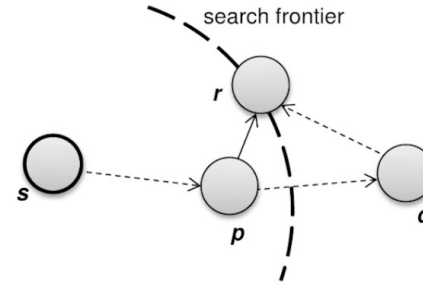
Need to change the stopping criterion:

- $|V| - 1$ iterations in the worst case
 - In practice $\sim$ graph diameter
- When all distances stay the same

Can use:

- External driver to check if the criterion is met
- Use Hadoop MapReduce Counters

Equivalent to the Bellman–Ford algorithm



MapReduce. PageRank

Uses the connectivity between sites to define page authority

- More inbound links —> more authority
- If an inbound link is from a page with high authority —> more authority

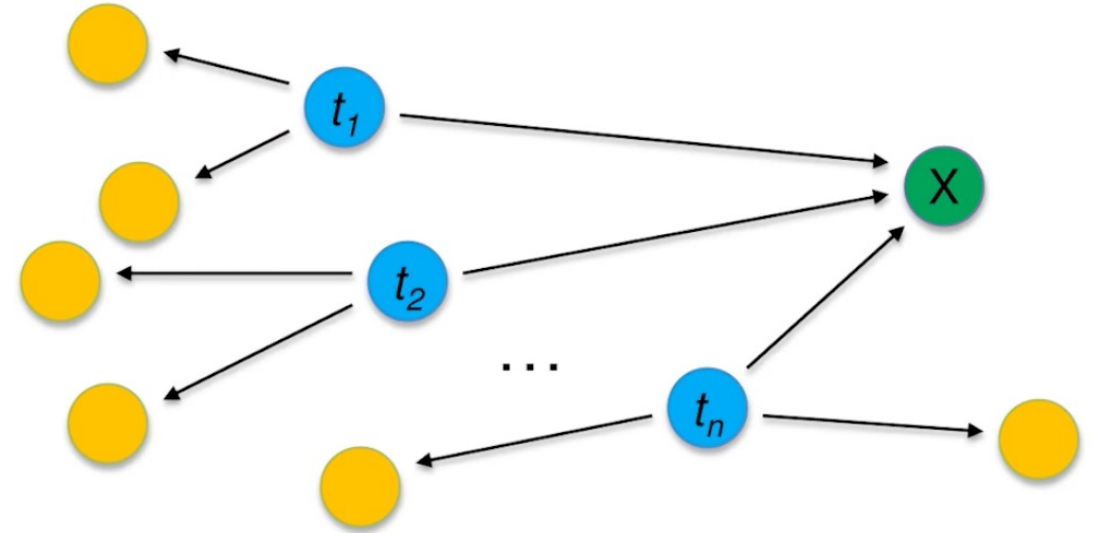
Defined as the average time a user will spend on the page when following a random walk:

1. User starts at a random page
2. User follows a random link

MapReduce. PageRank. Definition

Assume:

- All start pages are uniformly distributed
- All links are uniformly distributed
- With probability α can go to a random page



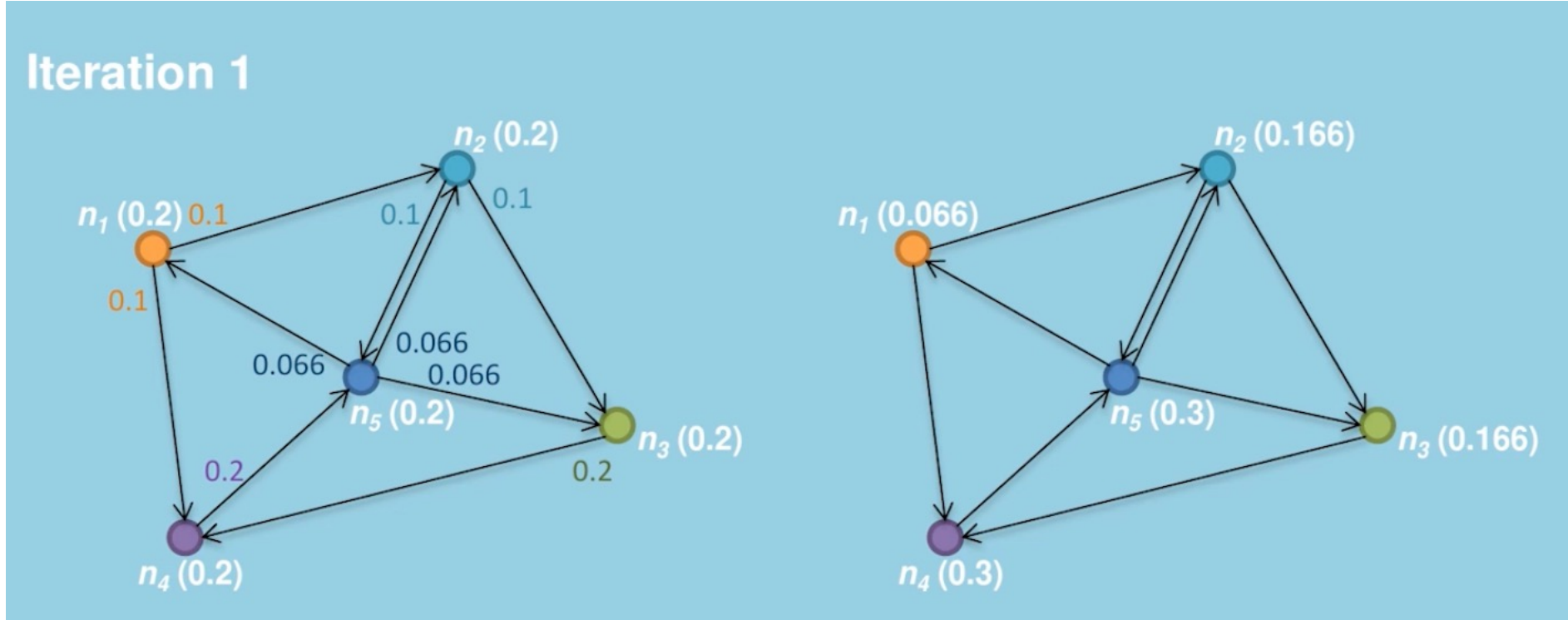
Then PageRank is defined as a stationary point of the equation:

$$PR(x) = \alpha \left(\frac{1}{N} \right) + (1 - \alpha) \sum_{i=1}^n \frac{PR(t_i)}{C(t_i)}$$

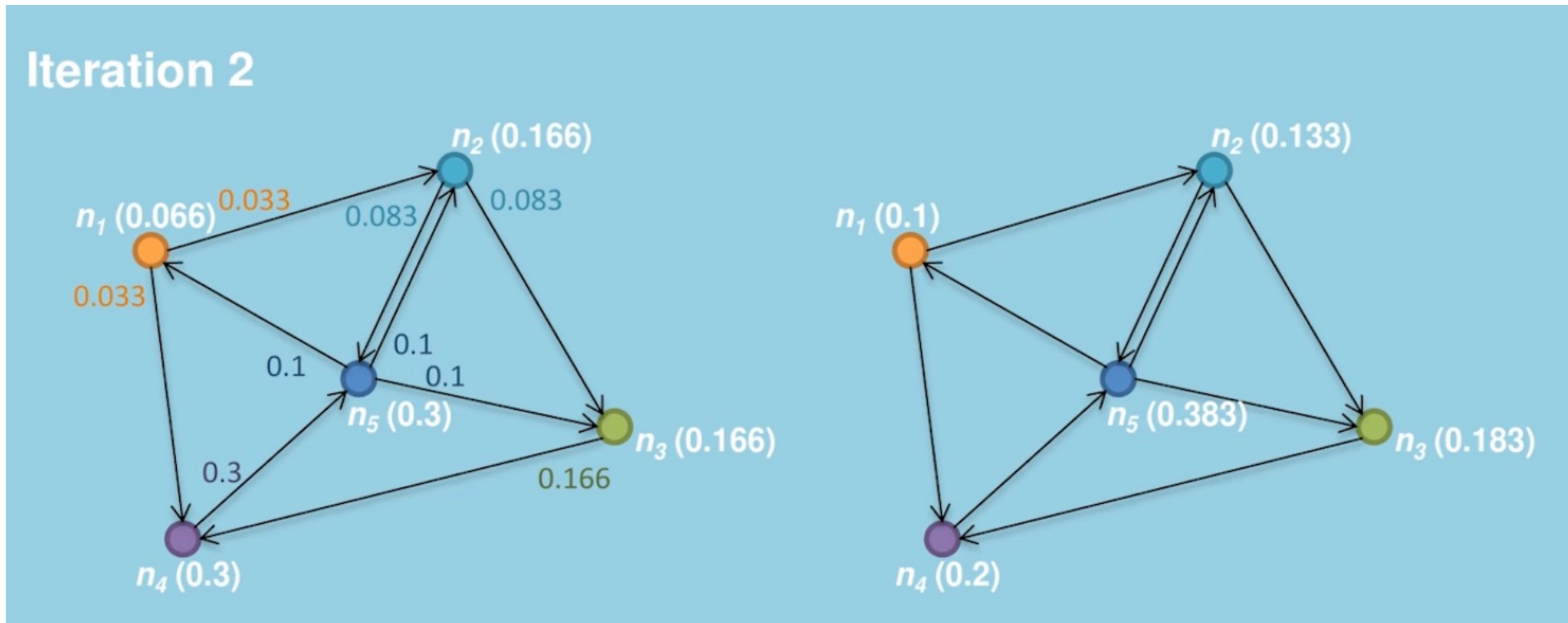
Can be evaluated using the simple iteration method

Usually: $\alpha \approx 0.1$

MapReduce. PageRank. Example



MapReduce. PageRank. Example



MapReduce. PageRank. Implementation

Repeats until:

- Convergence
- Fixed number of iterations
- Ignores the random jump
 - Can use an external driver
- Do not handle hanging nodes
 - Then the algorithm converges to a degenerate solution where all probability mass m ends up in the hanging nodes
 - Can use Counters and an external driver

The final formula:

$$p' = \alpha \left(\frac{1}{N} \right) + (1 - \alpha) \left(\frac{m}{N} + p \right)$$

```
# PageRank
def map(node_id, (PR, adj_list)):
    p = PR / len(adj_list)
    # Preserve the structure (scheme) of the data
    Emit(node_id, adj_list)
    for node_jd in adj_list:
        Emit(node_jd, p)

def reduce(node_id, [D1, ...])
    PR, adj_list = 0.0, None
    for d in [D1, ...]:
        if isinstance(d, list):
            adj_list = d
        else:
            PR += d

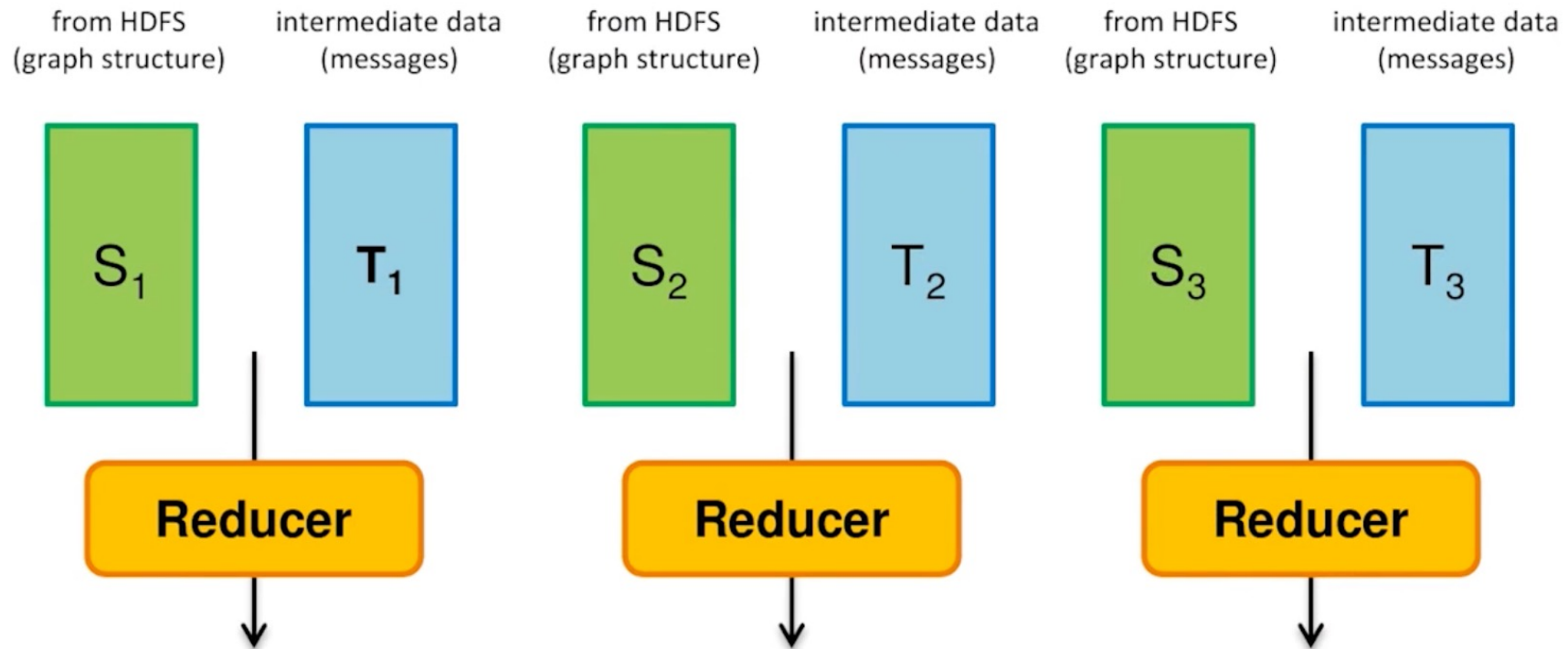
    Emit(node_id, (PR, adj_list))
```


MapReduce. Graph processing problems

- Iterative MapReduce algorithms are inefficient:
 - Hadoop task startup time
 - Slow or stuck tasks
- The uselessness of the shuffle phase for graphs:
 - Need to pass the graph structure from the mapper to the reducer
 - Use **In-mapper Combiners**
 - Use the **structure broadcasting**
- Checks at every iteration
- Graphs have a strongly connected local structure:
 - Better to reduce data in correspondence to locality —> **custom partitioning**

MapReduce. Schimmy Design Pattern

- Each reducer reads and outputs the graph structure
- Intermediate data and graph structure are consistently partitioned by the key



MapReduce. PageRank Performance

